

FORENSIC MEDICINE DEPARTMENT DATABASE SYSTEM

Equal Full-Stack Software Division & GitHub Contribution Plan (4 Members)

Evaluation Strategy: Equal GitHub Code Commits across Frontend, Backend & Database	Team Structure: 4 Full-Stack Members (Equal Workload)
Frontend Framework: React + Vite (HTML/CSS/JS)	Backend & Database: Node.js / Express (or Flask) + MySQL

1. Purpose of Equal Software Division for GitHub Evaluation

To ensure that course evaluators reviewing the group's GitHub repository see **equal commit history, code volume, and full-stack technical contribution** from all 4 team members, the software application is divided into **4 vertical feature modules**. Each member builds a complete end-to-end slice containing React Frontend Views, Express/Flask REST API Endpoints, MySQL Queries/Triggers, and Unit Tests.

2. Full-Stack Software Workload Distribution Matrix

Member	Feature Module Slices	Frontend React Components	Backend API Endpoints	SQL Tables & Triggers
Member 1	Auth, Security & Staff Administration	- Login.jsx & Signup.jsx - StaffManagement.jsx - RolePermissionManager.jsx	- POST /api/auth/login - GET/POST /api/staff - GET/POST /api/doctors - PUT /api/roles/permissions	- User, Role, RolePermission, Staff, Doctor - Password hash procedure
Member 2	Patient Intake, Case & Evidence Tracking	- PatientRegistration.jsx - Cases.jsx (Case Record UI) - Evidence.jsx (QR Code & Custody)	- GET/POST /api/patients - GET/POST /api/cases - GET/POST /api/evidence - POST /api/evidence/transfer	- Patient, CaseRecord, CaseHistory, Evidence, ChainOfCustody, LaboratoryTest - Case auto-number trigger
Member 3	Clinical Forensic Examinations & MLR	- MlefForm.jsx (Clinical exam) - MLReport.jsx (MLR generation) - ReferralsAndReviews.jsx	- GET/POST /api/mlef - GET/POST /api/mlr - PUT /api/mlr/issue - GET/POST /api/referrals	- MedicoLegalExamForm, MedicoLegalReport, Referral, ReviewAppointment - MLR lock trigger
Member 4	Autopsy, Court Summons & Audit/Notifications	- AutopsyForm.jsx (PM Autopsy) - CauseOfDeathForm.jsx - CourtSummons.jsx - AuditAndNotifications.jsx	- GET/POST /api/postmortem - GET/POST /api/cause-of-death - GET/POST /api/court-summons - GET /api/audit-logs	- Postmortem, CauseOfDeath, InquestOrder, CourtReport, CourtSummons, AuditLog, Notification, Document, Photograph, ReportTemplate - AuditLog trigger

3. Detailed Full-Stack Software Tasks & GitHub Commit Plan

MEMBER 1: Auth, Security & Staff Admin (Full-Stack Vertical)

Frontend Code (React): Build Login.jsx, Signup.jsx, StaffManagement.jsx, and RolePermissionManager.jsx. Implement JWT token persistence in localStorage, login state context, and permission-based UI routing.

Backend Code (API): Create authController.js and staffController.js routes. Implement bcrypt password hashing, token verification middleware (authMiddleware.js), and staff CRUD endpoints.

Database Code (SQL): Write SQL scripts for User, Role, RolePermission, Staff, Doctor tables. Write stored procedure for credential validation.

GitHub Commit Targets: Auth UI components, Auth controller API routes, JWT middleware, Staff/Doctor management components.

MEMBER 2: Patient Intake, Cases & Evidence Tracking (Full-Stack Vertical)

Frontend Code (React): Build PatientRegistration.jsx, Cases.jsx, and Evidence.jsx. Implement barcode/QR code rendering component using qrcode.react library for evidence samples.

Backend Code (API): Create patientController.js, caseController.js, and evidenceController.js. Build endpoints for patient creation, case intake assignment, evidence chain of custody transfers, and lab test status updates.

Database Code (SQL): Write SQL scripts for Patient, CaseRecord, CaseHistory, Evidence, ChainOfCustody, LaboratoryTest. Write trigger for auto-generating unique case numbers ('CW/2026/XXX').

GitHub Commit Targets: Patient reg UI, Case management routes, Evidence QR component, Chain of custody transfer API.

MEMBER 3: Clinical Forensic Exams & Medico-Legal Reports (Full-Stack Vertical)

Frontend Code (React): Build MlefForm.jsx, MlReport.jsx, and ReferralsAndReviews.jsx. Include interactive injury location selector and printable MLR report preview screen.

Backend Code (API): Create mlefController.js, mlrController.js, and referralController.js. Build routes for saving examination findings, finalizing MLRs, generating receipt certificates, and managing clinical referrals.

Database Code (SQL): Write SQL scripts for MedicoLegalExamForm, MedicoLegalReport, Referral, ReviewAppointment. Write trigger to prevent editing MLRs once status is 'Finalized'.

GitHub Commit Targets: MLEF form component, MLR generation controller, Referral tracking UI, PDF/Print report stylesheet.

MEMBER 4: Autopsy Component, Court Summons & Audit/Notifications (Full-Stack Vertical)

Frontend Code (React): Build AutopsyForm.jsx, CauseOfDeathForm.jsx, CourtSummons.jsx, and AuditAndNotifications.jsx. Include postmortem internal/external findings inputs and court appearance calendar.

Backend Code (API): Create autopsyController.js, courtController.js, and auditController.js. Build API routes for postmortem procedures, cause of death forms, summons tracking, and fetching immutable audit logs.

Database Code (SQL): Write SQL scripts for Postmortem, CauseOfDeath, InquestOrder, CourtReport, CourtSummons, AuditLog, Notification, Document, Photograph, ReportTemplate. Write automated MySQL AuditLog trigger for all table changes.

GitHub Commit Targets: Autopsy intake UI, Cause of Death API, Court summons calendar component, Audit log trigger script.

4. Recommended GitHub Commit Guidelines for Evaluator Audit

To ensure evaluators see strong, distinct contributions from all 4 members:

- **Individual Feature Branches:** Each member should work on their own git branch (e.g., feature/auth-m1, feature/cases-m2, feature/mlef-m3, feature/autopsy-m4).
- **Commit Frequency:** Every member should make 10-15 meaningful commits covering their assigned frontend components, backend controllers, and SQL files.
- **Descriptive Commit Messages:** Use structured commit prefixes (e.g., feat(auth): add JWT login controller, feat(cases): build patient registration form).
- **Pull Request Reviews:** Merge feature branches into main using Pull Requests so GitHub records code reviews by team members.